

Tutorial - Semana 6 - Encontro 2

Introdução

No Encontro 1, `DT_Items` passou a existir como tabela centralizada de itens, mas ainda usa o tipo padrão do editor para suas linhas — o que significa que qualquer campo pode receber qualquer valor, sem garantia de forma ou de categoria. Este encontro resolve esse problema com Struct e Enum: o Struct define a forma exata que cada linha da tabela deve ter, e o Enum restringe um campo a um conjunto fechado de categorias. Ao final, cada grupo aplica essa tabela tipada a um Actor de coleta concreto, reutilizando integralmente `BPI_Interactable` e o Event Dispatcher construídos na Semana 5 — e o encontro fecha com o Checkpoint de progresso do Módulo 2.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisitar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar Structs e Enums como organizadores de dados tipados dentro de uma Data Table.
- Modelar `S_ItemData` e `E_ItemType` e aplicá-los a `DT_Items`.
- Implementar um conjunto próprio de itens coletáveis aplicado a um Actor de coleta que reutiliza `BPI_Interactable` e Event Dispatcher.

Resultado Esperado ao Final da Semana

`DT_Items` tipada por `S_ItemData` e `E_ItemType`, populada com um conjunto próprio de itens coletáveis, e um Actor de coleta funcional no Vertical Slice que consulta essa tabela ao ser interagido, reutilizando `BPI_Interactable` e o Event Dispatcher da Semana 5. Resultado formalizado no Checkpoint de progresso do Módulo 2.

Pré-requisitos

- `DT_Items` criada na subpasta `Data/DataTables/` (Encontro 1).
 - `BPI_Interactable` e Event Dispatcher de interação funcionais (Semana 5).
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto do Encontro 1, com `DT_Items` criada e ao menos uma linha de exemplo preenchida.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo obrigatório. Meshes já disponíveis no projeto (ou primitivas simples, como cubo ou esfera) podem representar o Actor de coleta (`BP_Chest` ou `BP_Pickup`).

Projeto esperado

O projeto do Encontro 1, com `DT_Items` criada e o objeto interativo da Semana 5 (`BPI_Interactable` + Event Dispatcher) funcional no nível de teste.

Parte 1 — Struct e Enum: dando forma e categoria ao dado

Objetivo

Compreender Structs e Enums como organizadores de dados tipados e criar `S_ItemData` e `E_ItemType` , aplicando-os a `DT_Items` .

Conceito

Se a Data Table resolve "onde o dado mora", Struct e Enum resolvem "que forma esse dado deve ter". Sem tipagem, cada linha de `DT_Items` poderia receber qualquer valor em qualquer campo, e uma

categoria como "tipo de item" ficaria sujeita a erro de digitação — texto livre em vez de uma opção fechada.

Um **Struct** agrupa campos relacionados sob um único tipo nomeado. `S_ItemData`, por exemplo, reúne nome, descrição, ícone e valor em uma única estrutura, tornando `DT_Items` fortemente tipada linha a linha: toda linha passa a ter exatamente os mesmos campos, na mesma forma.

Um **Enum** restringe um campo a um conjunto fechado e nomeado de opções. `E_ItemType`, com valores como Consumível, Chave ou Recurso, elimina a ambiguidade de um campo de texto livre, permitindo que sistemas futuros — como o Inventário da Semana 10 — tomem decisões com base na categoria, não em comparação de strings.

Juntos, Struct e Enum transformam `DT_Items` de uma planilha solta em uma estrutura de dados confiável para o restante do projeto.

Passo a passo

1. No Content Browser, navegar até a subpasta `Data/Structs_Enums/` (criar caso ainda não exista, conforme `PROJECT_ARCHITECTURE.md`).
2. Clicar com o botão direito na subpasta e selecionar **Blueprints > Structure**.
3. Nomear a nova estrutura `S_ItemData`, conforme a convenção `S_`.
4. Abrir `S_ItemData` e adicionar os campos tipados: `Nome` (Text), `Descricao` (Text), `Icone` (Texture2D, como Soft Object Reference) e `Valor` (Integer ou Float).
5. Salvar `S_ItemData`.
6. Na mesma subpasta, clicar com o botão direito e selecionar **Blueprints > Enumeration**.
7. Nomear a nova enumeração `E_ItemType`, conforme a convenção `E_`.
8. Adicionar as categorias fechadas: `Consumivel`, `Chave`, `Recurso` (ou outras definidas em conjunto com a turma).
9. Salvar `E_ItemType`.
10. Adicionar um campo `Tipo` (do tipo `E_ItemType`) dentro de `S_ItemData`, e salvar novamente.
11. Abrir `DT_Items` (do Encontro 1) e, nas propriedades da tabela (Row Structure), trocar o tipo de linha para `S_ItemData`.
12. Repopular as linhas de `DT_Items` com os campos agora tipados, preenchendo `Tipo` com uma das categorias de `E_ItemType`.
13. Compilar e salvar todos os assets envolvidos.

Resultado esperado

`DT_Items` tipada por `S_ItemData`, com um campo `Tipo` restrito às categorias de `E_ItemType`, e todas as linhas preenchidas de forma consistente.

Verificando

Abrir `DT_Items` no Data Table Editor e confirmar que a coluna `Tipo` de cada linha apresenta um menu suspenso com as opções de `E_ItemType`, em vez de um campo de texto livre.

Problemas comuns

- **Criar campos redundantes ou não tipados:** por exemplo, um campo de texto livre para categoria em vez de usar `E_ItemType`, reproduzindo o mesmo problema que o Enum resolve. Pergunte a si mesmo: se outro colega do grupo digitar a categoria de forma ligeiramente diferente na próxima linha, o sistema perceberia isso como um erro? Se a resposta for não, o campo deve ser Enum, nunca texto livre.
- **Esquecer de trocar a Row Structure de `DT_Items` para `S_ItemData`:** sem essa troca, a tabela continua usando o tipo padrão do editor, e a tipagem criada não tem efeito algum sobre ela.
- **Colocar `S_ItemData` ou `E_ItemType` dentro de um Blueprint específico, em vez da subpasta `Data/Structs_Enums/`:** Structs e Enums usados por mais de um sistema devem ficar centralizados nessa subpasta, nunca duplicados dentro de um Actor.

Boas práticas

Mantenha `S_ItemData` com apenas os campos que qualquer item do Vertical Slice precisa ter em comum. Atributos muito específicos de um único tipo de item (por exemplo, um efeito exclusivo de uma poção rara) não pertencem ao Struct genérico — esse tipo de exceção é assunto para módulos futuros, não para este encontro.

Comparação com Unity

A Unity resolve o mesmo problema de tipagem com classes serializáveis (`[System.Serializable]` `class`) para o equivalente de um Struct, e com `enum` da própria linguagem C# para o equivalente de um Enum — o princípio de restringir a forma e as opções válidas de um dado é idêntico. A diferença arquitetural está na integração com a ferramenta de edição: na Unreal, o Struct definido em Blueprint já gera automaticamente as colunas correspondentes na Data Table e é editável nativamente no Data Table Editor, enquanto na Unity a exibição de uma classe serializável ou de um enum dentro do Inspector depende de como o campo é declarado e, em alguns casos, de atributos adicionais para aparecer de forma equivalente.

Parte 2 — Aplicando DT_Items a um Actor de coleta

Objetivo

Conectar `DT_Items` tipada a um Actor de coleta concreto do Vertical Slice, reutilizando `BPI_Interactable` e o Event Dispatcher da Semana 5.

Conceito

Uma Data Table tipada só tem valor se algum sistema efetivamente a consulta. O Actor de coleta (`BP_Chest` ou `BP_Pickup` , conforme `PROJECT_ARCHITECTURE.md`) é esse sistema: ele não guarda, em variáveis próprias, o nome ou a descrição do item que representa — ele guarda apenas um identificador de linha (Row Name) e, no momento da interação, consulta `DT_Items` por esse identificador para obter os dados completos. A lógica de "como reagir à interação" continua sendo a mesma da Semana 5 — implementar `BPI_Interactable` e disparar um Event Dispatcher —, mas agora essa reação carrega dado estruturado, em vez de ser apenas uma notificação vazia.

Passo a passo

1. Criar (ou reaproveitar) um Actor na subpasta `Blueprints/Interactables/` , nomeado `BP_Chest` ou `BP_Pickup` , conforme o tipo de coleta escolhido pelo grupo.
2. Garantir que esse Actor implementa `BPI_Interactable` (Class Settings > Implemented Interfaces), reaproveitando o padrão da Semana 5.
3. Criar, dentro do Actor, uma variável do tipo `Name` (ou `String`) chamada `ItemRowName` , usada como identificador da linha em `DT_Items` que este Actor representa.
4. Dentro da implementação da função `Interact` (herdada de `BPI_Interactable`), adicionar o nó **Get Data Table Row**, referenciando `DT_Items` e usando `ItemRowName` como chave de busca.
5. A partir do resultado de **Get Data Table Row** (uma instância de `S_ItemData`), disparar o Event Dispatcher criado na Semana 5, passando os dados obtidos (nome, ícone ou outro campo relevante) como parâmetro do Dispatcher, caso o grupo deseje que os inscritos recebam essa informação.
6. Compilar e salvar `BP_Chest` / `BP_Pickup` .
7. No nível de teste, posicionar ao menos uma instância do Actor de coleta e definir seu `ItemRowName` com o identificador de uma linha existente em `DT_Items` .

8. Testar em modo Play: interagir com o Actor de coleta e confirmar que a consulta à tabela ocorre corretamente, disparando o Event Dispatcher com os dados do item.
9. Preparar a apresentação do progresso do grupo para o Checkpoint ao final do encontro.

Resultado esperado

Um Actor de coleta funcional no nível de teste, implementando `BPI_Interactive`, consultando `DT_Items` pelo identificador de linha ao ser interagido, e disparando o Event Dispatcher da Semana 5 com os dados obtidos.

Verificando

Adicionar temporariamente um Print String conectado ao resultado de **Get Data Table Row**, exibindo o nome do item consultado, e confirmar em modo Play que o valor exibido corresponde exatamente à linha de `DT_Items` associada ao `ItemRowName` configurado. Remover o Print String antes da apresentação do Checkpoint.

Problemas comuns

- **Hardcodar os dados do item diretamente no Actor, ignorando `DT_Items`** : isso repete o problema que toda a semana resolveu; qualquer informação exibida deve vir da consulta à tabela, nunca de uma variável fixa redigitada no Actor.
- **`ItemRowName` não corresponder a nenhuma linha existente em `DT_Items`** : **Get Data Table Row** falha silenciosamente ou retorna valores padrão nesse caso; conferir a grafia exata do identificador contra as linhas da tabela.
- **Disparar o Event Dispatcher antes de obter o resultado de Get Data Table Row**: a consulta precisa ocorrer antes do disparo, para que os dados estejam disponíveis aos inscritos no momento da notificação.

Boas práticas

Mantenha o Actor de coleta enxuto: sua única responsabilidade é saber qual linha consultar e notificar via Event Dispatcher. Qualquer lógica de reação mais elaborada (efeito visual, atualização de inventário) pertence a quem se inscreve no Dispatcher, não ao Actor de coleta em si — o mesmo princípio de responsabilidade única já aplicado na Semana 5.

Comparação com Unity

O equivalente, em Unity, seria o Actor de coleta guardar uma referência a um `ScriptableObject` de item (ou um identificador para buscar em uma lista/array de `ScriptableObjects`), consultando seus campos no momento da interação, e notificando outros sistemas via `UnityEvent` ou `Action`. O princípio — Actor de coleta como consumidor de um dado externo, não como dono do dado — é o mesmo nas duas engines; a diferença está na forma de armazenamento e busca (linha de tabela por identificador na Unreal, referência direta a asset ou busca em lista na Unity).

Desafio

Cada grupo modela seu próprio conjunto de itens coletáveis (baús, moedas, recursos ou outra categoria própria) usando `DT_Items` tipada por `S_ItemData` e `E_ItemType`, com liberdade sobre categorias e atributos, desde que o Actor de coleta reutilize `BPI_Interactable` e o Event Dispatcher da Semana 5 para sinalizar a coleta.

Ao final da semana

Ao final da Semana 6 (Encontros 1 e 2), o Vertical Slice deve possuir `DT_Items` funcional, tipada por `S_ItemData` e `E_ItemType`, populada com um conjunto próprio de itens coletáveis, e um Actor de coleta (`BP_Chest` ou `BP_Pickup`) integrado que reutiliza `BPI_Interactable` e o Event Dispatcher da Semana 5 para sinalizar a coleta. Conceitualmente, a turma deve dominar a separação entre dado de design (Data Table, Struct, Enum) e lógica de gameplay (Interface, Event Dispatcher) como dois problemas distintos e complementares de arquitetura de motores. Isso corresponde à conclusão da linha "DT_Items (Data Table + Struct + Enum)" e ao início de "BP_Chest, BP_Pickup" no roadmap de `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2). O Checkpoint de progresso do Módulo 2 formaliza esse ponto de controle antes do encerramento da Unidade II na Semana 7.

Checklist

- ☐ `S_ItemData` criada em `Data/Structs_Enums/` com campos tipados (nome, descrição, ícone, valor, tipo)
- ☐ `E_ItemType` criada em `Data/Structs_Enums/` com categorias fechadas
- ☐ `DT_Items` com Row Structure trocada para `S_ItemData` e linhas repopuladas
- ☐ Actor de coleta (`BP_Chest` / `BP_Pickup`) implementando `BPI_Interactable`

- ☐ Consulta a `DT_Items` via `ItemRowName` funcionando corretamente na interação
- ☐ Event Dispatcher da Semana 5 disparado com os dados obtidos da tabela
- ☐ Nós de teste temporários (Print String) removidos do Blueprint final
- ☐ Checkpoint de progresso do Módulo 2 registrado

Glossário

- **Struct:** tipo customizado que agrupa campos tipados relacionados a uma mesma entidade de dado, usado aqui para definir a forma das linhas de `DT_Items`.
- **Enum:** conjunto fechado e nomeado de categorias possíveis para um campo, usado aqui para restringir o campo `Tipo` de `S_ItemData`.
- **Row Structure:** o Struct que define as colunas de uma Data Table.
- **Get Data Table Row:** nó de Blueprint que busca uma linha específica de uma Data Table a partir de um identificador (Row Name), retornando os dados dessa linha na forma do Struct associado.

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine (Data Assets e Data Tables). Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução a Data Tables, Structs e Enums. Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — ScriptableObject, classes serializáveis e enums em C#, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeos sugeridos (apenas como apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, com vídeos introdutórios de Data Tables e Structs; **Mathew Wadstein**, para explicações pontuais de WTF Is? Data Table e Struct.

Imagem sugerida

Objetivo: mostrar o fluxo completo de consulta de dado no momento da interação.

Enquadramento: diagrama de fluxo horizontal, três blocos conectados por setas. Elementos

importantes: bloco 1 — "Jogador interage" (ícone de Actor de coleta); bloco 2 — "Get Data Table Row com ItemRowName" (ícone de tabela); bloco 3 — "Event Dispatcher dispara com dados de S_ItemData" (ícone de sino ou notificação). O que deve ser destacado: a seta entre bloco 1 e bloco 2 representa a interface `BPI_Interactable`; a seta entre bloco 2 e bloco 3 representa a consulta tipada retornando `S_ItemData`. Legenda sugerida: "Do clique do jogador ao dado tipado: interação, consulta e notificação em três passos."